Автоматизация поставки: IaC, CI/CD и конвейер знаний

Платформа товарных знаний на базе мультимодального GraphRAG

Домашнее задание 8. Курс OTUS “ИИ-архитектор”

Содержание

Контекст **1**

1. Почему прежний отказ был неверен **2**

2. Инфраструктура как код **2**
2.1. Что поднимается на этапе 1 2
2.2. Целевая схема и условия перехода 4

3. Конвейер поставки кода **4**
3.1. Ручных действий ровно два 5

4. Конвейер знаний и стык с поставкой кода **6**
4.1. Единица выпуска - четвёрка, а не файл 6
4.2. Смена весов - самый дорогой вид выпуска 7

5. Стратегия выпуска **8**
5.1. Лестница: по аудитории и по числу запросов 8
5.2. Как переключается трафик 8
5.3. Метрики отката 10
5.4. Откат 11

6. Состояние: что спроектировано и что входит в объём этапа 1 **11**

7. Вывод **12**

Контекст

Репозиторий проекта: https://github.com/SandalAndrey/otus_ai_architect

Заказчик - интернет-магазин коллекционных масштабных моделей: более 15 000 позиций, около 400 марок, порядка 600 производителей. Система отвечает на товарные вопросы покупателей и сотрудников, работает в закрытом контуре без обращений к внешним LLM-API.

Материалы работы в репозитории:

- docs/16-delivery-pipeline.md - полное описание;
- ADR-0020 - решение о стратегии выпуска;
- три схемы в docs/diagrams/: delivery-cicd, delivery-mlops, delivery-canary.

Оговорка о статусе. Это проект поставки, а не отчёт о внедрении. Файлы в репозитории есть - infra/Dockerfile, infra/compose.yml, infra/terraform/, infra/release/, .github/workflows/ci.yml, - но это скелеты: terraform apply не выполнялся ни разу, образ не собирался, стек не поднимался, из пяти заданий конвейера работает одно. Код приложения появляется с работы 3.1 плана работ. Состояние по каждому механизму - в разделе 6.

Работа началась с противоречия. Проект дважды отказался от канареечного развёртывания - в плане проверки качества и в документе о наблюдаемости, - одной и той же формулировкой: “при 30 000 запросов в месяц статистика не наберётся”. Задание требует ровно того, от чего проект отказался. Разбор этого противоречия оказался содержательнее, чем сама схема конвейера, поэтому с него и начну.

1. Почему прежний отказ был неверен

Формулировка смешала две разные вещи, и верна она только для одной.

Канарейка как эксперимент отвечает на вопрос “новая версия лучше или хуже”. Это сравнение средних, ему нужна мощность выборки. При 30 000 запросов в месяц доля в 1 процент даёт около 10 запросов в сутки, и ответа не будет никогда. Здесь прежний отказ верен.

Канарейка как ограничитель радиуса поражения отвечает на другой вопрос: “не сломалось ли”. Статистики он не требует вовсе, если сигнал выбран так, что его ожидаемое значение равно нулю. Ответ без ссылки на источник недопустим по NFR-2.4 в принципе, а не “чаще, чем раньше”. Одно событие - уже вердикт, и для вердикта хватает десяти запросов.

Прежний отказ распространил вывод о первом на второе. Это ошибка рассуждения, а не ошибка в числах. Формулировки в обоих документах исправлены, решение вынесено в ADR-0020.

2. Инфраструктура как код

2.1. Что поднимается на этапе 1

По ADR-0009 этап 1 - один арендованный хост с GPU и замкнутый контур на нём. Kubernetes на одном узле даёт накладные расходы без единой выгоды, которой он платит: ни масштабирования, ни отказоустойчивости узла, ни сложной маршрутизации здесь нет. Поэтому код делится на два уровня: Terraform поднимает облачные ресурсы, cloud-init и Compose разворачивают на них стек.

Фрагмент ниже - сокращённая выдержка из infra/terraform/main.tf, который **ни разу не применялся**: модуль хранилища не написан, значения переменных не заданы. Он фиксирует состав ресурсов и принятые решения; подробнее о состоянии - в разделе 6.

```
provider "yandex" { zone = "ru-central1-a" }   # РФ: требование NFR-3.5
```

```
resource "yandex_vpc_network" "core" { name = "assistant-net" }

resource "yandex_vpc_subnet" "edge" {          # только обратный прокси
  network_id     = yandex_vpc_network.core.id
  v4_cidr_blocks = ["10.10.1.0/24"]
}

resource "yandex_vpc_subnet" "internal" {      # хранилища и инференс
  network_id     = yandex_vpc_network.core.id
  v4_cidr_blocks = ["10.10.2.0/24"]
}

resource "yandex_compute_instance" "inference" {
  platform_id = "gpu-standard-v3"
  resources { cores = 8, memory = 96, gpus = 1 }   # A100 40 ГБ, ADR-0019

  boot_disk {
    initialize_params { image_id = var.gpu_image_id, size = 200 }
  }

  network_interface {
    subnet_id = yandex_vpc_subnet.internal.id
    nat       = false          # выхода в интернет нет на уровне маршрутизации
  }

  metadata = { user-data = file("cloud-init.yaml") }
}

module "object_storage" {                      # зоны озера из ADR-0011
  source     = "./modules/minio"
  buckets    = ["bronze", "silver", "gold", "traces", "backups"]
  versioning = { bronze = true, silver = false, gold = false }
  retention  = { traces = "30d", backups = "90d" }
}

resource "yandex_lockbox_secret" "app" { name = "assistant-secrets" }
```

Три решения в этом коде существеннее самого кода.

Выход в интернет отсутствует на уровне маршрутизации. Требование закрытого контура записано в проекте с самого начала, но держалось на том, что в коде нет обращений к внешним API. Это намерение, а не механизм: достаточно одной библиотеки, которая пойдёт за обновлением. Отсутствие NAT у служебной подсети делает требование проверяемым.

Состояние Terraform хранится в объектном хранилище с блокировкой. Иначе два применения с разных машин расходятся, и инфраструктура перестаёт описываться кодом фактически, оставаясь описанной формально.

Секретов в коде нет. Terraform создаёт секрет и права на него, значения кладутся отдельно: состояние Terraform содержит значения ресурсов в открытом виде.

Стек на узле - одиннадцать сервисов в Compose: обратный прокси, приложение, vLLM, Neo4j, Qdrant, MinIO, Dagster и контур наблюдаемости из четырёх контейнеров. Версии образов закреплены тегами, а не `latest`: иначе перезапуск контейнера меняет версию без следа в истории.

2.2. Целевая схема и условия перехода

Отказ от Kubernetes - решение этапа 1, а не принцип.

Что меняется	Проект этапа 1	Целевая схема
Оркестрация	Compose на одном узле	Managed Kubernetes, группа узлов с GPU
Объектное хранилище	MinIO в контуре	Оно же: облачный S3 нарушил бы закрытый контур
Маршрутизация	Вес на обратном прокси	Ingress и штатный разбор трафика
Инференс	Один контейнер vLLM	Группа подов с автомасштабированием

Условия перехода, любое из двух: нагрузка превышает возможности одной карты по расчёту из ADR-0019, либо появляется второй инференс-узел. Переход до этого - накладные расходы без выгоды, и записывать его в смету этапа 1 значит её завышать.

Переносимость обеспечивается тем, что приложение упаковано в образы и не зависит от способа их запуска: при переходе Compose-файл заменяется на чарт, а код приложения, конвейер CI и состав гейтов не меняются.

3. Конвейер поставки кода

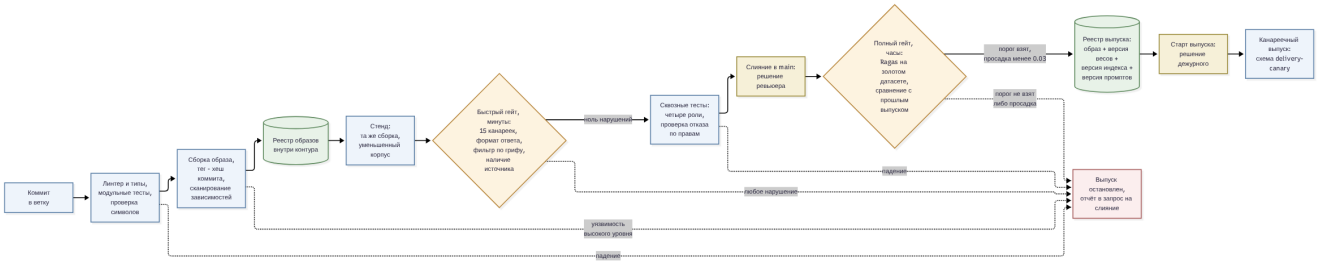


Рис. 1: Конвейер поставки кода: от коммита через линтер, модульные тесты, сборку образа и стенд к двум гейтам и реестру выпуска. Жёлтым - два единственных ручных действия. Исходник - `delivery-cicd.mmd`, картинка в полном разрешении

Этап	Что делает	Время
Линтер и типы	Статический разбор, проверка символов в документации	Секунды

Этап	Что делает	Время
Модульные тесты	Нормализация наименований, чанкинг, разбор прайсов	Минуты
Сборка образа	Тег - хеш коммита, сканирование зависимостей	Минуты
Стенд	Та же сборка, уменьшенный корпус	Минуты
Быстрый гейт	15 канареек, формат ответа, фильтр по грифу, наличие источника	Минуты
Сквозные тесты	Четыре роли, проверка отказа по правам	Минуты
Полный гейт	Ragas на золотом датасете, сравнение с прошлым выпуском	Часы

Разделение гейтов по времени - не удобство, а следствие замера: одна пара вопрос-ответ стоит судье 123 секунды при трёх метриках, на 50 парах это часы. Поэтому между выпусками система проверяется канарейками, а качество меряется реже, чем меняется код.

Раннер стоит внутри контура: судьёй служит локальная модель, и золотой датасет за периметр не уходит. Версия интерпретатора на нём закреплена явно - Ragas тянет зависимости, которые отстают от свежих версий Python, и обновление интерпретатора способно уронить оценку, не затронув наш код. Это не предположение: при замере стоимости судьи так и произошло.

3.1. Ручных действий ровно два

Действие	Кто	Почему не автоматизировано
Слияние в main	Ревьюер	Решение о содержании изменения. Автослияние по зелёным проверкам означает, что код не читают
Старт выпуска	Дежурный	Выбор момента: выпуск не начинают в пятницу вечером и в час пиковой нагрузки

Всё остальное - сборка, проверки, развёртывание на стенде, продвижение по ступеням, откат по жёсткому сигналу - без участия человека. Единственное исключение в сторону ручного: отказ судьи на полном гейте разбирается инженером, потому что судья ошибается.

4. Конвейер знаний и стык с поставкой кода

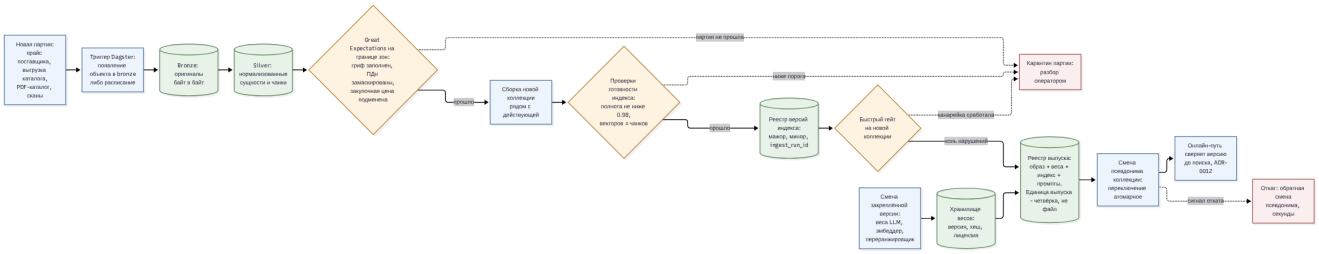


Рис. 2: Конвейер знаний: партия документов, триггер Dagster, зоны озера с проверками качества, сборка новой коллекции рядом с действующей, реестр версий индекса. Смена версии весов входит в тот же реестр выпуска. Исходник - `delivery-mlops.mmd`, картинка в полном разрешении

Обучения модели в проекте нет - это записано в конвейере данных и повторено в документе о наблюдаемости: модели не обучаются и не дообучаются, разделения train/val нет, дрейфовать нечему. Типовая цепочка принимает такой вид.

Типовой шаг	Что на его месте у нас
Обновился датасет	Пришла партия: прайс поставщика, выгрузка каталога, PDF
Триггер оркестратора	Dagster по появлению объекта в bronze либо по расписанию
Переобучение	Переиндексация: разбор, извлечение сущностей, векторизация
Проверки обучения	Great Expectations на границах зон, проверки готовности индекса
Model Registry	Реестр версий индекса плюс хранилище весов
Запуск CI/CD	Быстрый гейт на новой коллекции, затем смена псевдонима

Подмена не косметическая. В обучаемой системе артефакт - файл весов, и его качество меряется на отложенной выборке. У нас артефакт - коллекция векторов и граф, а качество меряется тем же золотым датасетом, что и качество кода.

4.1. Единица выпуска - четвёрка, а не файл

Отсюда главный вывод работы: **индекс и код нельзя выпускать по отдельности**. Выпускается закреплённый набор.

Составляющая	Чем версионируется
Код приложения	Хеш коммита, он же тег образа
Веса моделей	Версия и хеш файла, лицензия
Индекс	Мажор и минор коллекции, `ingest_run_id`
Промпты и схемы агентов	Версия в репозитории, внутри образа

Реестр выпуска хранит эти четыре идентификатора вместе с отчётами обоих гейтов. Это и есть связь между кодом приложения и артефактом модели: по записи реестра восстанавливается, какой код на каких весах и на какой версии индекса отвечал пользователю в конкретный день.

Запись - один файл на выпуск, каталог `infra/release/`. Записей там пока нет: они создаются последним шагом полного гейта, вручную не пишутся. Лежит образец формата, сокращённо:

```yaml
release: "2026-11-14-a7f3c21"      # значение метки release в метриках

code:
  image: "registry.internal/assistant:a7f3c21"
  commit: "a7f3c21e94..."
  prompts_version: "1.4"

weights:
  llm:      { name: "Qwen3-14B", quant: "INT4", sha256: "...", license: "Apache-2.0" }
  embedder: { name: "bge-m3",                   sha256: "...", license: "MIT" }
  reranker: { name: "bge-reranker-v2-m3", sha256: "...", license: "Apache-2.0" }

index:
  collection: "catalog_v4_2"
  major: 4                         # сверяется онлайн-путём до поиска, ADR-0012
  minor: 2
  ingest_run_id: "dagster-2026-11-13-0417"

gates:
  fast: { passed: true, canaries: "15/15" }
  full: { passed: true, faithfulness: 0.86, relevancy: 0.91, delta: -0.01 }

supersedes: "2026-10-30-3c1b8d9"   # держится горячей 72 часа
```

Поле `release` не декоративное: его значение становится меткой на всех метриках, и по нему канарейка отделяется от базы.

Основание для неразделимости - ADR-0012: расхождение между преобразованиями на индексации и в онлайн-пути уже опознано в проекте как дефект (по природе это то же, что training-serving skew), и выпуск кода отдельно от индекса - прямой способ его получить. Онлайн-путь поэтому сверяет мажорную версию коллекции до поиска, а не после.

4.2. Смена весов - самый дорогой вид выпуска

- Смена эмбеддера обесценивает весь индекс: векторы несопоставимы, нужна полная переиндексация, а не инкрементальная.
- Смена весов LLM требует полного гейта целиком: поведение меняется во всём.
- Канарейки для весов нет по причине из раздела 5.2.

Поэтому смена весов выполняется blue-green в окне плановых работ за счёт бюджета ошибок, а не канареечно.

5. Стратегия выпуска

5.1. Лестница: по аудитории и по числу запросов

При 30 000 запросов в месяц это около 1000 запросов в сутки. Один процент - десять запросов: ступень, на которой нечего измерять и остаётся только ждать. Поэтому лестница не 1 - 5 - 100.

Ступень	Аудитория	Запросов в сутки	Условие перехода
0. Тень	Никто, ответы не отдаются	-	200 запросов, окно GPU
1. Внутренний контур	Все сотрудники	по факту	100 запросов и 1 рабочий день
2. Публичный, 5 процентов сессий	Покупатели	около 50	200 запросов и 2 рабочих дня
3. Публичный, 25 процентов сессий	Покупатели	около 250	500 запросов и 2 рабочих дня
4. Публичный, 100 процентов	Все	1000	Прежняя версия горячей 72 часа

Две особенности против обычной лестницы.

Первая ступень - аудитория, а не доля. Сотрудники во внутреннем контуре сообщают о странном ответе словами, а не метрикой. При нашем объёме это ценнее любой доли процента: сто запросов от человека, который знает каталог, информативнее тысячи анонимных.

Условие перехода - число запросов и время одновременно. Запросы без времени могут набраться за утренний всплеск и не показать вечернего профиля; время без запросов не проверяет ничего.

5.2. Как переключается трафик

Доля задаётся весом в конфигурации обратного прокси, сессия закрепляется за версией по хешу идентификатора диалога.

Закрепление обязательно, а не желательно: по NFR-1.6 диалог хранит до 60 обменов контекста, и сессия, перескочившая между версиями на середине, получит ответ, не связанный с предыдущим. Пользователь увидит дефект продукта, а не аккуратный выпуск.

Service mesh не ставится: на одном хосте с четырьмя контейнерами он добавил бы задержку в бюджет 5 секунд и слой, который некому сопровождать.

Канарейка на весах модели невозможна, и причина не в статистике. По ADR-0019 скорость генерации равна полосе памяти, делённой на размер весов. Две модели в карте помещаются (10.1 + 10.1 + 7.5 ГБ кэша = 27.7 из 40), но проходы по памяти делятся между ними: вместо 108 проходов в секунду каждая версия получает 54 при требуемых 90 токенах в секунду на поток. Канарейка на один процент трафика уронила бы скорость базовой версии вдвое и нарушила NFR-1.2 для всех пользователей, а не для одного процента.

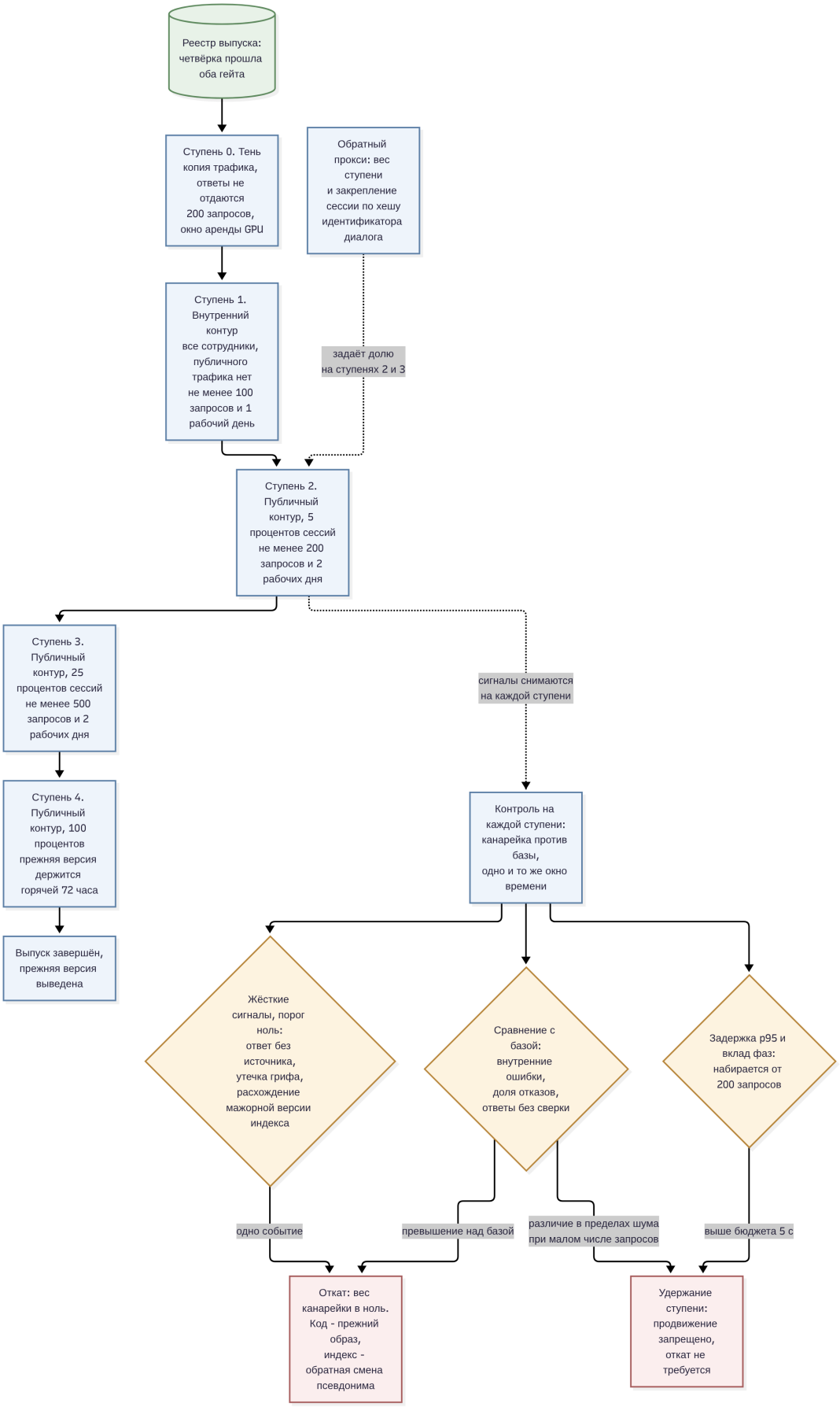


Рис. 3: Ступени канареечного выпуска от теневого трафика до полного переключения, условия перехода по числу запросов и времени, три группы сигналов: жёсткие с нулевым порогом, сравнительные относительно базы и сигналы удержания. Исходник

Это и есть ответ на вопрос “почему не канарейка для модели” числом, а не словом “дорого”.

5.3. Метрики отката

Сигналы делятся на два класса, и смешивать их нельзя.

Жёсткие: порог ноль, откат автоматический. Эти метрики в исправной системе равны нулю всегда, поэтому вердикт выносится по одному событию и не требует ни сравнения с базой, ни набора статистики. Именно они делают канарейку возможной на нашем объёме.

Сигнал	Метрика	Основание
Ответ без ссылки на источник	`answers_without_citation_total`	NFR-2.4
Утечка атрибутов закрытого грифа	`acl_leak_total`	NFR-3.1, блокирующий критерий приёмки
Расхождение мажорной версии индекса	`index_version_mismatch_total{severity="major"}`	ADR-0012

Сравнительные: порог относительно базы в том же окне времени.

Сигнал	Условие отката
Внутренние ошибки и таймауты	3 и более на канарейке при нуле на базе
Доля отказов	Превышение базы вдвое при не менее чем 50 запросах
Ответы без сверки	Доля выше 3 процентов, порог ADR-0010

База измеряется одновременно с канарейкой, а не по вчерашним значениям. В этом единственное статистическое преимущество канарейки на малом объёме: суточный профиль, состав вопросов и состояние железа у двух версий общие, поэтому различие между ними содержательнее, чем различие с историей.

Условие, без которого это не работает: метка `release` на каждой метрике. Обе версии отвечают на одни и те же вопросы, с одного хоста, в одну секунду, - разделить их иначе нечем. Значение берётся из поля `release` записи реестра выпуска:

```
sum by (release) (rate(assistant_errors_total{type=~"internal|timeout"}[15m]))
  / sum by (release) (rate(assistant_requests_total[15m]))
```

Метка вводится вместе с реестром выпуска, до первой ступени лестницы. В первой редакции документа о наблюдаемости она стояла среди отложенного вместе со ступенями 2 и 3 - это была ошибка: без метки откат не работает ни на одной ступени, включая первую. Формулировка исправлена.

Цена метки - рост числа временных рядов. Ограничивается правилом: одновременно живут не более двух значений `release`, прежнее удаляется через 72 часа после завершения выпуска.

Сигналы удержания: продвижение запрещено, откат не запускается. Задержка p95 набирается от 200 запросов и на ступени 1 недостоверна. Рост доли деградации по NFR-6 штатен, а не является отказом.

Что сигналом не является. Метрики судьи - faithfulness и answer relevancy - в продуктиве не считаются: судья стоит десятки секунд на пару. Их место - полный гейт до выпуска. Судья, останавливающий выпуск сам себя, - плохая идея по той же причине, по которой judge-метрики не сделаны блокирующими.

5.4. Откат

Что откатывается	Как	Время
Код и промпты	Вес канарейки в ноль, прежний образ уже запущен	Секунды
Индекс	Обратная смена псевдонима коллекции	Секунды
Веса модели	Возврат прежней версии и перезапуск инференса	Минуты, цена - холодный старт

Прежняя версия держится горячей 72 часа после полного переключения: срок выбран так, чтобы пережить выходные - дефект на редком классе вопросов за двое суток может не встретиться.

6. Состояние: что спроектировано и что входит в объём этапа 1

Файлы есть, работающего конвейера нет. В репозитории лежат скелеты.

Файл	Состояние
`infra/Dockerfile`	Не собирается: каталога `backend/app` не существует
`infra/compose.yml`	Не поднимается: образ не собирается, значения не заданы
`infra/terraform/`	Не применялся: модуль хранилища не написан
`infra/release/`	Записей нет, лежит образец формата
`.github/workflows/ci.yml`	Работает 1 задание из 5, остальные отключены `if: false`

Скелеты зафиксированы раньше кода намеренно: решения о сегментации сети, о способе хранения состояния Terraform и о том, что попадает в образ, дешевле принять до написания приложения, чем подгонять под него.

Работающее задание конвейера - проверка символов в документации. Выбрано не случайно: в проекте принято писать только основными символами клавиатуры, а правило, которое проверяется глазами, нарушается на третьей неделе. Сейчас 66 файлов репозитория проходят проверку.

Таблица отвечает на вопрос “входит ли механизм в объём работ этапа 1”, а не “сделан ли он”.

Механизм	Объём этапа 1	Основание
Terraform и cloud-init для узла и хранилищ	Входит	Работа 3.1
Конвейер CI с обоими гейтами	Входит	Работы 4.1 и 4.4
Реестр выпуска как четвёрка	Входит	Записи в метаданных, часов почти не требует
Метка `release` на метриках	Входит	Предусловие сравнения канарейки с базой
Blue-green индекса сменой псевдонима	Входит	Предусмотрен ADR-0012, работа 3.3
Ступени 1 и 4 лестницы	Входит	Опирается на разделение контуров из НФТ
Ступени 2 и 3: доля сессий	Отложено	Требует весов на прокси и закрепления сессии
Ступень 0: теневой трафик	Отложено	Требует второй копии сервиса и окна GPU
Автоматический откат по жёстким сигналам	Отложено	Алерты описаны, автоматизация действия - нет
Kubernetes	Отложено	Условия перехода в разделе 2.2

Смета этапа 1 не пересматривается: то, что входит в объём, опирается на уже заложенные работы.

Порядок реализации задан зависимостями: среда и образы, затем конвейер CI, затем реестр выпуска вместе с меткой `release`, затем гейты, и только после этого ступени лестницы. Третий шаг часто пытаются отложить “до первой канарейки” - это ошибка, метка нужна уже на ступени 1.

Следует назвать своим именем. Когда ступени 2 и 3 будут отложены при реализации, лестница сведётся к внутреннему контуру и полному переключению, то есть окажется ближе к blue-green, чем к канарейке. Изображать полную схему как действующую было бы неверно вдвойне: она не только не работает, её ещё и не начинали строить.

7. Вывод

Задание требовало показать связь кода с артефактом модели, безопасность выпуска и минимум ручных действий. По каждому пункту работа дала не то, что подразумевалось в типовой схеме.

Связь кода и артефакта. Реестра моделей в обычном смысле у нас быть не может: модель не обучается. Его место занял реестр выпуска как четвёрка “код - веса - индекс - промпты”, и это оказалось строже типовой схемы: там артефакт модели версионируется отдельно от кода, у нас они неразделимы по требованию консистентности индексации.

Безопасность выпуска. Прежний отказ от канарейки снят, но не целиком: канарейка как эксперимент на нашем объёме по-прежнему невозможна, а как ограничитель радиуса - работает, потому что три ключевые метрики имеют нулевой порог. Канарейка на весах модели закрыта расчётом полосы памяти.

Автоматизация. Ручных действий в спроектированной схеме два, и оба - решения, а не операции.

Чего работа не доказывает. Пороги сравнительных сигналов (“три ошибки при нуле на базе”) - инженерное суждение, подобранное так, чтобы ложное срабатывание было редким; выведены они не из данных и проверяются только эксплуатацией. Числа ступеней лестницы опираются на гипотезу о 30 000 запросов в месяц, которая сама ещё не подтверждена Заказчиком.

И главное: **из описанного работает одно задание конвейера из пяти.** Скелеты файлов в репозитории фиксируют решения, но не проверяют их: `terraform apply` не запускался, образ не собирался, ни одной канареечной выкладки не выполнялось. Это проект поставки, а не отчёт о внедрении. Проверка начинается на работах 3.1 и 4.5, и там же выяснится, какие из принятых здесь решений придётся пересмотреть.